Examination Management Portal - Project Report

1. Student details

Name	Mayank Kumar Poddar	**Roll number**	23f3004197
Email	23f3004197@ds.study.iitm.ac.in	**Course / term**	MAD-1, September 2026 term

2. Project details

Problem: Institutions manage exam schedules, examiner availability, slot booking and evaluation with spreadsheets and emails. Build a web app where an **Admin**, **Examiners** and **Students** work through their own roles.

Approach: I split the problem into three simple ideas and kept the code to match.

- **Data:** seven tables (see ER diagram). One `User` table with a role covers admin, examiner and student.
- **Rules in one place:** a `services/` folder holds every business rule (slot windows, capacity, duplicates, who may evaluate). Web routes only read forms and show pages, so each rule can be tested without a browser.
- **Safety by design:** overbooking is stopped by one atomic SQL `UPDATE ... WHERE booked < capacity`; duplicate bookings are stopped by a unique database index; roles are enforced once per blueprint; exam status is calculated from dates plus admin switches, so it cannot get out of sync.

Features: admin dashboard with counts and charts; course, exam and rubric management; examiner approval; open/close slot creation and booking; reschedule bookings and change examiner; search by name or ID; examiner slots, rubric-based marking and remarks; student search/filter, booking, cancellation, history, results, PDF scorecard; JSON API; email notifications; paged lists; light/dark theme; 18 automated tests; Docker files.

Design: an "academic editorial" interface (Fraunces and Schibsted Grotesk fonts, ivory and ink palette with a vermilion accent) built only with Bootstrap plus one custom stylesheet. Bookings are shown as admission tickets; scores as CSS-only rings. It is responsive (sidebar becomes a slide-out menu on phones), keyboard friendly, and meets WCAG AA colour contrast in both themes. Demo data uses real IIT Madras BS course codes and names.

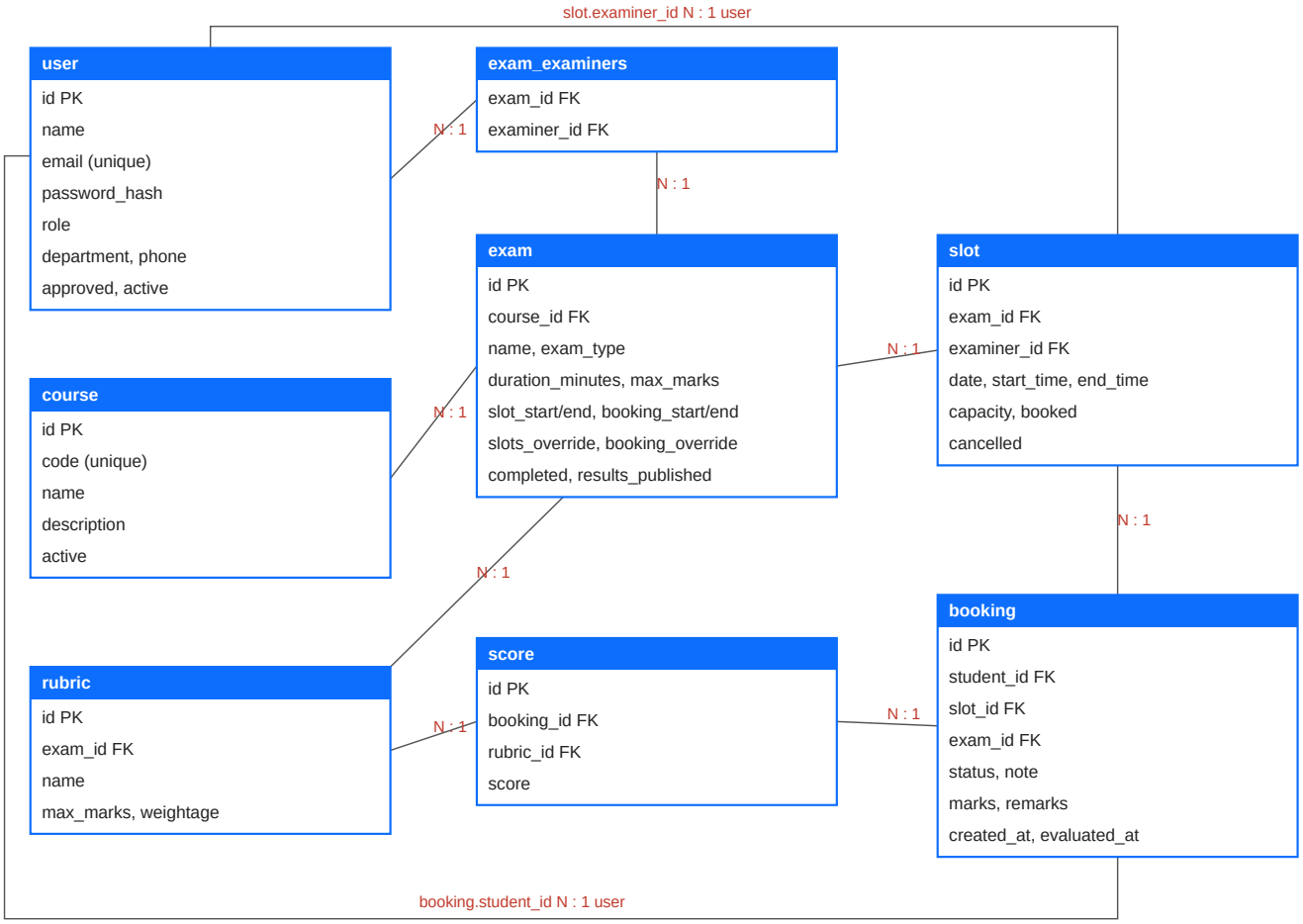
3. AI / LLM declaration

I used Claude Code (Anthropic's AI coding assistant) extensively in this project. Working from my decisions on the structure, libraries, features and design, it generated most of the code, templates, tests, demo data and documentation, produced the demo video, and helped find and fix bugs. I reviewed each module, ran the application and the tests, and I can explain and modify any part of it.

4. Frameworks and libraries

Flask, Jinja2, Flask-SQLAlchemy (SQLite), Flask-Login, Flask-WTF (CSRF only), Flask-Mail, ReportLab (PDF scorecard), Bootstrap 5 (CSS and JS, saved locally), Chart.js (optional dashboard charts only), Google Fonts, pytest, gunicorn. Core features use no JavaScript; JS is used only for the theme switch, the collapsible sidebar, tooltip placement, the mobile menu and the optional charts.

5. ER diagram



Notes: `booking.exam_id` and `score.rubric_id` are also foreign keys (exam and rubric). `exam_examiners` links exams to assigned examiners (many-to-many). `booking` keeps cancelled rows for full history; a partial unique index allows only one live booking per student and exam. `slot.booked` counts taken seats.

6. API resource endpoints (JSON, login session)

Method	Path	Who	Purpose
GET	/api/exams, /api/exams/<id>	any user	list (search with ?q=) / one exam
GET	/api/exams/<id>/slots	any user	slots of an exam
GET	/api/examiners, /api/students	admin	people lists
GET	/api/bookings	any user	own bookings (all for admin)
POST	/api/bookings {"slot_id": n}	student	book a slot
DELETE	/api/bookings/<id>	student	cancel before deadline

7. Presentation video

[PASTE DRIVE LINK HERE]